Architectural Integration and Source-Level Implementation of Support for the GFX1012 Microarchitecture within the ROCm Unified Software Stack

The persistent challenge of deploying high-performance compute workloads on consumer-grade AMD hardware, specifically the Radeon RX 5500 series utilizing the Navi 14 silicon (internally designated as the gfx1012 microarchitecture), stems from a misalignment between the official Radeon Open Compute (ROCm) support matrix and the underlying capabilities of the Heterogeneous-compute Interface for Portability (HIP). When a system administrator or developer encounters the error message from the HIP runtime stating "No compatible code objects found for: gfx1012:xnack-," it signals a terminal failure in the code object loading phase, where the dispatch mechanism cannot locate a binary segment within the application's loaded shared objects that matches the Instruction Set Architecture (ISA) of the physical hardware.[1] To resolve this and establish functional support for gfx1012, one must conduct a comprehensive audit of the fatbinary distribution across the ROCm math libraries and re-engineer the kernel generation pipeline managed by the Tensile toolchain.

Analysis of the Binary Dispatch Failure and Code Object Metadata

The evidence provided by the initial system audit using the strings utility reveals a critical deficit in the pre-compiled binaries provided by standard ROCm distributions. The presence of exactly three occurrences of the gfx1012 string in librocblas.so and libhipblaslt.so, contrasted with zero occurrences in libhipblas.so and libtorch_hip.so, indicates that while some metadata stubs for Navi 14 exist in the low-level BLAS provider, the actual kernel implementations and the higher-level framework wrappers are entirely devoid of support for this architecture.[4]

The Role of hip_fatbin.cpp in the ROCm Execution Pipeline

The failure at hip_fatbin.cpp:687 is indicative of a mismatch during the fatbinary registration process. In the ROCm environment, a fatbinary (or "fatbin") is a container format that aggregates multiple code objects, each compiled for a different GPU architecture. When a HIP-based application, such as PyTorch, initializes, the runtime must iterate through these fatbinaries to select the code object that corresponds to the current device's ISA.[1]

Feature	GFX1012 Target Status	Implications for Loading
Microarchitecture	RDNA 1.0 (Navi 14)	Requires specific ISA-v3 or ISA-v4 code objects.
XNACK Support	Disabled (xnack-)	Mandatory flag for consumer RDNA hardware; mismatch causes load failure.[7]
SRAM ECC	Disabled (sram-ecc-)	Must match the hardware configuration reported by rocminfo.
Fatbinary Presence	Missing in libtorch_hip.so	PyTorch-native kernels cannot execute on the device.[8]

The logic within hip_fatbin.cpp utilizes a matching algorithm that compares the hardware's architecture string (e.g., amdgcn-amd-amdhsa--gfx1012) against the "bundles" available in the shared objects. The logs showing "Forcing SPIRV: false" and "value of HIP_FORCE_SPIRV_CODEOBJECT: 0" suggest the system is attempting a native ISA dispatch rather than a platform-independent intermediate representation.[1] Consequently, the lack of a matching ISA segment in libtorch_hip.so prevents even basic tensor operations from offloading to the GPU, leading to the observed runtime errors.

Interpreting the Strings Count Disparity

The investigator's observation that librocblas.so contains three instances of gfx1012 while libhipblas.so contains zero is explained by the hierarchical nature of the ROCm math stack. hipBLAS serves as a thin dispatch layer that abstracts the backend (either rocBLAS on AMD or cuBLAS on NVIDIA).[4] Therefore, libhipblas.so often contains no architecture-specific kernels, relying instead on dynamic linking to its backend provider.

The three hits in librocblas.so are typically not actual GPU kernels but rather metadata stubs used for "lazy loading." In modern ROCm releases, the actual kernels for rocBLAS are often moved into external data files (e.g., TensileLibrary_lazy_gfx1030.dat) to prevent the primary .so file from reaching impractical sizes.[10] The presence of only three strings suggests that while rocBLAS is aware of the architecture name, it does not actually contain the generated kernels for gfx1012.[3]

The Tensile Kernel Factory: Engineering Source-Level

Support

The fundamental prerequisite for matrix multiplication on AMD GPUs is the Tensile library. Tensile is a complex Python-based toolchain that generates assembly kernels specifically tuned for the hardware characteristics of a given GPU architecture, such as the number of Compute Units (CUs), the wavefront size, and the memory controller width.[13]

Architectural Similarities and the Navi 10 Fallback

The gfx1012 (Navi 14) microarchitecture is a derivative of the gfx1010 (Navi 10) architecture. Both share the RDNA 1.0 instruction set, though they differ in the physical scale of the silicon. Because Navi 10 is more frequently supported in community builds, a viable engineering path for gfx1012 involves adapting the Tensile "Logic" files designed for gfx1010.[13]

To enable this at the source level, one must modify the Tensile/Common.py file within the Tensile source tree. This file defines the architectureMap and SupportedISA parameters.

1. **ISA Identification**: Ensure that the tuple (10, 1, 2) is added to the list of supported Instruction Set Architectures.
2. **Logic Mapping**: In the architectureMap dictionary, create an entry that maps gfx1012 to a directory containing functional RDNA 1.0 assembly logic.
3. **Assembly Capabilities**: In the CACHED_ASM_CAPS section, the capabilities of gfx1012 must be defined. Since RDNA 1 does not support certain newer operations like Dot8 or Matrix-Fused-Multiply-Add (MFMA) found in CDNA architectures, the configuration must strictly adhere to the navi10 capability set.[12]

The mathematical efficiency of these kernels is governed by the block size and tile parameters. For a General Matrix Multiply (GEMM) operation, the complexity of the kernel tuning can be expressed by the relationship:

$$W_{utilization} = \frac{M_{tile} \times N_{tile}}{CU_{count} \times Wavefront_{size}}$$

If the CU_count for gfx1012 (which is 22 or 24 CUs depending on the variant) is not properly accounted for in the Tensile logic, the generated kernels may exhibit poor performance or result in "invalid device function" errors due to incorrect workgroup sizing.[19]

Modifying rocBLAS and hipBLAS Repositories

The investigator’s find command reveals the structure of the hipBLAS repository. To add support, one must look specifically at the CMake configuration files. The AMDGPU_TARGETS variable in the top-level CMakeLists.txt or CMakePresets.json is the primary gatekeeper for architecture support.[7]

When building rocBLAS, the following configuration must be enforced:

-DAMDGPU_TARGETS="gfx1012" This triggers Tensile to generate kernels specifically for Navi 14. If the build system fails with a "ModuleNotFoundError" related to Tensile, it indicates that the internal virtual environment used by rocBLAS for kernel generation has not been correctly initialized, a common issue in ROCm source builds.[22]

Rebuilding the PyTorch Stack for GFX1012

Even if the BLAS libraries are successfully patched, PyTorch will still fail to execute non-linear operations if libtorch_hip.so is not recompiled. PyTorch contains its own set of HIP kernels (written in C++ using templates) for functions such as ReLU, Adam optimizer, and Batch Normalization. These are compiled using hipcc during the PyTorch installation process.[8]

The PYTORCH_ROCM_ARCH Environment Variable

To force the PyTorch build system to include gfx1012 support, the PYTORCH_ROCM_ARCH environment variable must be set prior to running the setup script. Standard wheels from pytorch.org are typically built for gfx906, gfx90a, gfx1030, and gfx1100, which is why the investigator's pre-built installation shows zero hits for gfx1012.[8]

The sequence for a successful build is:

1. **Clone the Repository**: git clone --recursive https://github.com/pytorch/pytorch.git
2. **Define Architecture**: export PYTORCH_ROCM_ARCH=gfx1012
3. **Execute Build**: python setup.py install.[17]

This process invokes the hipcc compiler for every .hip and .cu file in the repository, embedding the compiled Navi 14 ISA into the resulting libtorch_hip.so fatbinary. This can be verified post-build using: strings libtorch_hip.so | grep gfx1012 A successful build should yield thousands of hits, representing each unique kernel compiled for the architecture.[11]

Addressing the xnack- Flag in Fatbinaries

A frequent source of the "No compatible code objects found" error is a subtle mismatch in feature flags. Consumer RDNA hardware like the RX 5500 does not support XNACK (page retry).[2] If the compiler defaults to xnack+ or if the fatbin contains an ambiguous target, the loader in hip_fatbin.cpp will reject it. The build system must be explicitly instructed to target gfx1012:xnack- to ensure compatibility with the driver's report of the hardware state.[1]

Runtime Overrides and Diagnostic Strategies

For users who cannot rebuild the entire stack, the ROCm environment provides a mechanism to "alias" one architecture to another at the runtime level via the HSA (Heterogeneous System Architecture) override.[10]

The HSA_OVERRIDE_GFX_VERSION Mechanism

By setting export HSA_OVERRIDE_GFX_VERSION=10.1.1, the investigator can trick the ROCm loader into treating the gfx1012 card as a gfx1011 or gfx1010 device.[16] This is particularly useful because gfx1010 is more likely to have stubs in pre-compiled libraries. However, this override is only effective if the loaded libraries contain kernels for the *spoofed* version.[6]

Environment Variable	Recommended Value for GFX1012	Logical Result
HSA_OVERRIDE_GFX_VERSION	10.1.0	Masks device as Navi 10 (Standard RDNA 1).
HSA_OVERRIDE_GFX_VERSION	10.1.1	Masks device as Navi 12 (Pro 5600M).
HIP_VISIBLE_DEVICES	0	Isolates the target GPU for debugging.[13]
AMD_LOG_LEVEL	4	Provides verbose trace of kernel launch failures [User Query].

It should be noted that if multiple GPUs are present, the global override may cause instability in the other devices. Community reports indicate that per-device overrides (e.g., HSA_OVERRIDE_GFX_VERSION_0) are frequently non-functional in the current ROCm runtime, necessitating a single-GPU configuration for consistent results.[27]

Diverging from hipBLAS for Deep Code Investigation

The investigator's query about grepping for the error within the hipBLAS repository identifies a common misunderstanding of the ROCm architecture. The error "No compatible code objects found for" is generated by the HIP runtime itself, specifically within the clr (Common Language Runtime) or the hipamd repository.[1] hipBLAS merely calls into the HIP runtime; it does not manage the fatbinary loading logic.[5]

To "dive deep" into the logic, the investigator should clone the following repositories:

1. **ROCm/clr**: To examine hip_fatbin.cpp and hip_code_object.cpp. This is where the ISA matching logic resides.
2. **ROCm/rocBLAS**: To examine the Tensile integration and the handle.cpp file, which manages the backend math initialization.[13]

3. **ROCm/Tensile**: To examine Common.py, which is the ultimate source of truth for which architectures are "known" to the code generator.[29]

Engineering Solutions for Tensile-Logic Mismatch

A primary reason for the "3 strings" result in librocblas.so is that Tensile may be configured to build for gfx1012 but lacks the optimized assembly logic to produce actual kernels. In such cases, Tensile produces a "fallback" library that contains only basic, unoptimized stubs.[12]

Implementing the Fallback Logic Patch

A known issue in Tensile versions 5.x and 6.x is the failure to properly link fallback kernels for architectures without optimized logic files. This often leads to "invalid device function" errors when a GEMM operation is called. A critical engineering step involves patching TensileCreateLibrary.py to ensure it correctly maps gfx1012 to the navi10 assembly logic folder if a specific navi14 folder is absent.[12]

The patch typically involves:

1. Locating the architectureMap in the Python source.
2. Ensuring that the gfx1012 key points to the navi10 logic.
3. Modifying Tensile/Common.py to set globalParameters["IgnoreAsmCapCache"] = True, which prevents the build system from using cached (and potentially incorrect) capability data for the unknown architecture.[12]

Verification via Kernel Extraction

To definitively confirm that the generated libraries contain the necessary support, the investigator should use the extract-kernel tool provided in the ROCm binary directory. This tool can deconstruct a fatbinary and list the embedded ISAs.

```
cat /opt/rocm/lib/librocblas.so | /opt/rocm/bin/extract-kernel
```

If this command does not output a segment labeled amdgcn-amd-amdhsa--gfx1012, then no amount of environment variable tweaking will enable GPU acceleration for that library.[1]

Future Outlook and Sustainability of GFX1012 Support

As ROCm continues to evolve, official support for RDNA 1 (including gfx1012) is increasingly deprecated in favor of RDNA 2 (gfx103x) and RDNA 3 (gfx11xx).[25] This makes the ability to build from source and apply community patches essential for extending the lifecycle of Navi 14 hardware in AI and machine learning contexts.

Comparative Support Across ROCm Releases

ROCm Version	Official GFX1012 Status	Primary Conflict
ROCm 5.7	Supported in some distros	Last version with broad RDNA 1 stubs.[13]
ROCm 6.0	Community support required	Polaris/Vega removal; RDNA 1 deprioritized.[33]
ROCm 6.2	Legacy / Unsupported	Tensile logic requires manual mapping.[34]
ROCm 7.x	Deprecated	Strict adherence to CDNA/RDNA 3+.[25]

The investigator's environment (ROCm 6.2 on Arch Linux) is particularly susceptible to these issues, as Arch Linux's rolling release model often pulls the latest ROCm versions where legacy support has been stripped.[6] Maintaining functional gfx1012 support in this environment requires a persistent local build of rocBLAS and PyTorch with the specified architectural flags.[6]

Summary of System Engineering Actions

To achieve full integration of the gfx1012 architecture, the following workflow is recommended:

1. **Repository Acquisition**: Clone the Tensile, rocBLAS, and PyTorch repositories at versions compatible with the installed ROCm runtime.[4]
2. **Tensile Patching**: Manually add gfx1012 to Tensile/Common.py and map it to navi10 logic.[12]
3. **Math Library Rebuild**: Use rmake.py with AMDGPU_TARGETS="gfx1012" to generate Navi 14 specific BLAS kernels.[4]
4. **Framework Rebuild**: Compile PyTorch with PYTORCH_ROCM_ARCH=gfx1012 to ensure the core HIP kernels are present in libtorch_hip.so.[8]
5. **Validation**: Use AMD_LOG_LEVEL=4 and strings to verify that kernel launches no longer trigger the "No compatible code objects found" error in hip_fatbin.cpp.[1]

By shifting the focus from hipBLAS (the wrapper) to rocBLAS and Tensile (the generators), and by ensuring the libtorch_hip.so fatbinary is fully populated, the investigator can overcome the limitations of the pre-compiled software stack and enable full GPU acceleration on Navi 14 hardware.

Nuanced Conclusions on Hardware-Software

Co-Optimization

The integration of gfx1012 highlights a broader trend in the ROCm ecosystem where consumer-grade "gaming" hardware requires significant manual intervention to operate in "compute" environments. The mathematical complexity of GEMM kernels means that simple ISA compatibility is often insufficient; true performance requires kernels that are specifically tuned for the cache hierarchy and memory bandwidth of the Navi 14 chip. While the HSA_OVERRIDE_GFX_VERSION provides a quick fix, the sustainable path involves deep source-level modifications to the Tensile pipeline, ensuring that every layer of the stack—from the low-level ISA stubs in librocblas.so to the high-level tensor operations in libtorch_hip.so—is explicitly aware of and optimized for the gfx1012 microarchitecture. This approach not only resolves the immediate fatbinary loading error but also ensures that the hardware can operate at its peak theoretical throughput, effectively bridging the gap between consumer silicon and professional-grade software requirements.

The discrepancy in the strings analysis performed by the investigator serves as a definitive roadmap for this effort. The absence of architecture-specific data in the core HIP libraries of PyTorch is the primary bottleneck. By following the re-engineering steps outlined for the Tensile factory and the PyTorch build system, support for Navi 14 can be established, allowing for the execution of complex models like MobileNet v3 on hardware that would otherwise remain idle in a ROCm-based environment. This process exemplifies the open-source nature of the ROCm stack, which, despite its official support limitations, provides the necessary tools for developers to extend its functionality to a wider array of AMD hardware.

Obras citadas

1. ROCm 6.x appears Cannot find CO in the bundle libhipblaslt.so for ..., fecha de acceso: marzo 30, 2026, https://github.com/pytorch/pytorch/issues/127169
2. OpenVX GPU - HIP Backend - Gated HIP Launch · Issue #1437 · ROCm/MIVisionX - GitHub, fecha de acceso: marzo 30, 2026, https://github.com/ROCm/MIVisionX/issues/1437
3. Any plans for adding gfx10+ support? · Issue #648 · ROCm/hipBLASLt - GitHub, fecha de acceso: marzo 30, 2026, https://github.com/ROCm/hipBLASLt/issues/648
4. Installing and building for Linux — hipBLAS 3.4.0 Documentation, fecha de acceso: marzo 30, 2026, https://rocm.docs.amd.com/projects/hipBLAS/en/develop/install/Linux_Install_Guide.html
5. Building and Installing — hipBLAS 2.2.0 Documentation, fecha de acceso: marzo 30, 2026, https://rocm.docs.amd.com/projects/hipBLAS/en/docs-6.2.1/install.html
6. RuntimeError: HIP error: shared object initialization failed (rx 6800) (#10) · Issue - GitLab, fecha de acceso: marzo 30, 2026, https://gitlab.archlinux.org/archlinux/packaging/packages/python-pytorch/-/issues/10

7. Home · likelovewant/ollama-for-amd Wiki - GitHub, fecha de acceso: marzo 30, 2026, https://github.com/likelovewant/ollama-for-amd/wiki
8. PyTorch on ROCm installation, fecha de acceso: marzo 30, 2026, https://rocm.docs.amd.com/projects/install-on-linux/en/latest/install/3rd-party/pytorch-install.html
9. Building and Installing - hipBLAS documentation - Read the Docs, fecha de acceso: marzo 30, 2026, https://hipblas.readthedocs.io/en/latest/install.html
10. LocalAI ROCm Runtime Issue: HSA_OVERRIDE_GFX_VERSION Required -gfx1151 #5822, fecha de acceso: marzo 30, 2026, https://github.com/mudler/LocalAI/issues/5822
11. ROCm Linux packaging approaching bloatware: 29GB #4224 - GitHub, fecha de acceso: marzo 30, 2026, https://github.com/ROCm/ROCm/issues/4224
12. Tensile won't produce backend libraries for archs without optimized logic files when using --separate-architectures · Issue #1757 - GitHub, fecha de acceso: marzo 30, 2026, https://github.com/ROCm/Tensile/issues/1757
13. ZLUDA · vladmandic/sdnext Wiki - GitHub, fecha de acceso: marzo 30, 2026, https://github.com/vladmandic/sdnext/wiki/ZLUDA
14. Home · likelovewant/ROCmLibs-for-gfx1103-AMD780M-APU Wiki ..., fecha de acceso: marzo 30, 2026, https://github.com/likelovewant/ROCmLibs-for-gfx1103-AMD780M-APU/wiki
15. ZLUDA · vladmandic/sdnext Wiki - GitHub, fecha de acceso: marzo 30, 2026, https://github.com/vladmandic/sdnext/wiki/ZLUDA/e5179375d69a6363692e734911e0c76f31d67029
16. Re: Notes about ROCm on RDNA2 - Debian Mailing Lists, fecha de acceso: marzo 30, 2026, https://lists.debian.org/debian-ai/2022/07/msg00052.html
17. Regression in rocm 5.3 and newer for gfx1010 · Issue #2527 - GitHub, fecha de acceso: marzo 30, 2026, https://github.com/ROCm/ROCm/issues/2527
18. Build failure: Ollama in 24.05 Stable · Issue #316919 · NixOS/nixpkgs - GitHub, fecha de acceso: marzo 30, 2026, https://github.com/NixOS/nixpkgs/issues/316919
19. REQUEST: gfx803 support · Issue #173 · lamikr/rocm_sdk_builder - GitHub, fecha de acceso: marzo 30, 2026, https://github.com/lamikr/rocm_sdk_builder/issues/173
20. Adding -O0 causes HIP error: the operation cannot be performed in the present state · Issue #126 · ROCm/clr - GitHub, fecha de acceso: marzo 30, 2026, https://github.com/ROCm/clr/issues/126
21. pytorch: consistent HIPBLAS_STATUS_ALLOC_FAILED on 7900 XTX · Issue #1708 · ROCm/TheRock - GitHub, fecha de acceso: marzo 30, 2026, https://github.com/ROCm/TheRock/issues/1708
22. [Bug]: Error when compiling "ModuleNotFoundError: No module named 'Tensile.TensileCreateLibrary'" · Issue #1296 · ROCm/rocBLAS - GitHub, fecha de acceso: marzo 30, 2026, https://github.com/ROCm/rocBLAS/issues/1296
23. build error with install.sh -icd is it due to pip version · Issue #1234 · ROCm/rocBLAS - GitHub, fecha de acceso: marzo 30, 2026, https://github.com/ROCm/rocBLAS/issues/1234
24. how to resolve CMake error · Issue #1226 · ROCm/rocBLAS - GitHub, fecha de

acceso: marzo 30, 2026, https://github.com/ROCm/rocBLAS/issues/1226
25. [Issue] All nightly builds are failing due to PyTorch trying to build for all archs #3432 - GitHub, fecha de acceso: marzo 30, 2026, https://github.com/ROCm/TheRock/issues/3432
26. Regression in rocm 5.3 and newer for gfx1010 #4030 - GitHub, fecha de acceso: marzo 30, 2026, https://github.com/ROCm/ROCm/discussions/4030
27. Hardware support - Ollama's documentation, fecha de acceso: marzo 30, 2026, https://docs.ollama.com/gpu
28. HSA_OVERRIDE_GFX_VERSIO, fecha de acceso: marzo 30, 2026, https://github.com/ollama/ollama/issues/8473
29. rocm-libraries/shared/tensile/Tensile/Common.py at develop - GitHub, fecha de acceso: marzo 30, 2026, https://github.com/ROCm/rocm-libraries/blob/develop/shared/tensile/Tensile/Common.py
30. ROCm for RDNA2 - 6900XT / GFX1030 coming - Reddit, fecha de acceso: marzo 30, 2026, https://www.reddit.com/r/ROCm/comments/ouwasq/rocm_for_rdna2_6900xt_gfx1030_coming/
31. [Solved] Ollama-rocm not offloading the workload to gpu / Kernel & Hardware / Arch Linux Forums, fecha de acceso: marzo 30, 2026, https://bbs.archlinux.org/viewtopic.php?id=311125
32. Building ROCm 7.1 and PyTorch on Windows for Unsupported GPUs: My Hands-On Guide, fecha de acceso: marzo 30, 2026, https://medium.com/@guinmoon/building-rocm-7-1-and-pytorch-on-windows-for-unsupported-gpus-my-hands-on-guide-0758d2d2b334
33. Complete Guide How to run Pytorch with AMD Radeon GPU (gfx803) : r/ROCm - Reddit, fecha de acceso: marzo 30, 2026, https://www.reddit.com/r/ROCm/comments/1hrwrkr/complete_guide_how_to_run_pytorch_with_amd_radeon/
34. offloaded 0/35 layers to GPU on gfx1103 · Issue #12303 · ollama/ollama - GitHub, fecha de acceso: marzo 30, 2026, https://github.com/ollama/ollama/issues/12303
35. [Issue]: RX 7600 XT ROCm Torch Segmentation Fault Linux #4380 - GitHub, fecha de acceso: marzo 30, 2026, https://github.com/ROCm/ROCm/issues/4380
36. ROCm consolidated changelog, fecha de acceso: marzo 30, 2026, https://rocm.docs.amd.com/en/latest/release/changelog.html
37. ROCm consolidated changelog - ROCm Documentation - AMD, fecha de acceso: marzo 30, 2026, https://rocm.docs.amd.com/en/docs-7.0.0/release/changelog.html
38. python-torchvision-rocm CMake build error (solved: build in chroot) - Arch Linux Forums, fecha de acceso: marzo 30, 2026, https://bbs.archlinux.org/viewtopic.php?id=294557